

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 14 · Expresiones regulares y procesamiento de texto

Clases Pattern y Matcher

Cuantificadores, clases de caracteres y anclas

matches() frente a find() y grupos de captura

Validación de DNI, email, teléfono y fechas

replaceAll, split y procesamiento de texto real

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Regex y texto

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué clase compila una expresión regular en Java?
- a) Pattern
 - b) Regex
 - c) Matcher
 - d) RegexCompiler

RESPUESTA / SOLUCIÓN

a) Pattern — `Pattern.compile("...")` compila la regex; `Matcher` se obtiene con `pattern.matcher(texto)`.

-
2. ¿Qué método comprueba si TODA la cadena coincide con la expresión?
- a) `matches()`
 - b) `find()`
 - c) `lookingAt()`
 - d) `contains()`

RESPUESTA / SOLUCIÓN

a) `matches()` — Exige que la expresión cubra la cadena completa (equivale a anclar con `^` y `$`).

-
3. ¿Qué cuantificador significa 'una o más veces'?
- a) `*`
 - b) `+`

- c) ?
- d) {1}

RESPUESTA / SOLUCIÓN

b) + — a+ acepta 'a', 'aa', 'aaa'...; * acepta cero o más; ? cero o una.

-
4. ¿Qué método de Matcher busca la siguiente coincidencia dentro de la cadena?
- a) find()
 - b) next()
 - c) match()
 - d) search()

RESPUESTA / SOLUCIÓN

a) find() — Avanza por la cadena localizando coincidencias parciales; se repite mientras devuelva true.

-
5. En el código fuente Java, para representar la regex de un dígito hay que escribir "\\d" (doble barra).

RESPUESTA / SOLUCIÓN

Verdadero — La barra invertida es carácter de escape en los literales String, por lo que se necesita `\\` para que la regex reciba `\\d`.

6. ¿Qué expresión valida un teléfono de 9 dígitos que empieza por 6, 7 o 9?

- a) `[679]\\d{8}`
- b) `\\d{9}`
- c) `[6-9]{9}`
- d) `6|7|9\\d{8}`

RESPUESTA / SOLUCIÓN

a) `[679]\\d{8}` — Primera cifra 6, 7 o 9, seguidas de exactamente 8 dígitos.

7. ¿Qué método de String reemplaza todas las coincidencias de una regex?

- a) `replaceAll()`
- b) `replace()`
- c) `replaceFirst()`
- d) `replaceAllMatches()`

RESPUESTA / SOLUCIÓN

a) `replaceAll(regex, reemplazo)` — Sustituye todas las apariciones; `replace()` (sin regex) reemplaza literales.

8. ¿Qué construcción permite capturar una parte de la coincidencia para recuperarla después?

- a) (grupo)
- b) [grupo]
- c) {grupo}
- d) <grupo>

RESPUESTA / SOLUCIÓN

a) (grupo) — Los paréntesis crean grupos; con `matcher.group(1)` se recupera el primero.

9. La regex `^abc$` solo coincide con cadenas que empiecen y terminen exactamente en 'abc' (coincidencia completa).

RESPUESTA / SOLUCIÓN

Verdadero — `^` ancla el inicio y `$` el final; con `matches()` las anclas son redundantes porque ya exige la cadena completa.

10. ¿Qué método de `String` divide la cadena usando una expresión regular como separador?

- a) `split()`
- b) `cut()`
- c) `divide()`
- d) `tokenize()`

RESPUESTA / SOLUCIÓN

a) `split(regex)` — Devuelve un `String[]`; p. ej. `"a;b;c".split(";")` → `["a", "b", "c"]`.

Parte 2: Análisis y depuración

3 puntos

1. Este validador de emails acepta direcciones incorrectas como 'a@b.com.extra'.
¿Por qué y cómo se corrige? (1,5 pts)

```
String email = "a@b.com.extra";
Pattern p = Pattern.compile("[a-z]+@[a-z]+\\.com");
Matcher m = p.matcher(email);
if (m.find()) System.out.println("Email válido");
```

RESPUESTA / SOLUCIÓN

find() busca coincidencias parciales: 'a@b.com' aparece dentro de 'a@b.com.extra', así que el resultado es válido aunque la cadena completa no lo sea.

Corrección: usar matches() en lugar de find() (o anclar la regex con ^...\$). Además, el punto en \.com está bien escapado, pero el dominio puede incluir más niveles.

-
2. El siguiente código intenta extraer los precios de un texto, pero no imprime nada.
¿Qué falla? (1,5 pts)

```
String texto = "Pan 1,20 € - Leche 0,85 €";
Pattern p = Pattern.compile("(\\d+,\\d{2})");
Matcher m = p.matcher(texto);
if (m.find()) {
    System.out.println(m.group(1));
}
```

RESPUESTA / SOLUCIÓN

El código solo imprime UNA coincidencia (la primera: '1,20') porque usa if en lugar de while.

Corrección: sustituir if (m.find()) por while (m.find()) para recorrer todas las coincidencias e imprimir también '0,85'.

Parte 3: Ejercicios de programación

6 puntos

1. Escribe un programa que valide un NIF: 8 dígitos seguidos de una letra, y que la letra sea la correcta según el algoritmo (resto de dividir el número entre 23 con la tabla TRWAGMYFPDXBNJZSQVHLCKE). (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;

public class ValidarNIF {

    static final String LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("NIF: ");

        String nif = sc.nextLine().trim().toUpperCase();

        if (!nif.matches("\\d{8}[A-Z]")) {

            System.out.println("Formato incorrecto");

            return;

        }

        int num = Integer.parseInt(nif.substring(0, 8));

        char esperada = LETRAS.charAt(num % 23);

        if (nif.charAt(8) == esperada) System.out.println("NIF válido");

        else System.out.println("Letra incorrecta, debería ser " + esperada);

    }

}
```

2. Escribe un programa que, dado un texto, extraiga e imprima todas las palabras que empiezan por vocal (a, e, i, o, u), usando Pattern y Matcher con find() en bucle. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.regex.*;

public class Vocales {

    public static void main(String[] args) {

        String texto = "El oso y el águila comen uvas en la isla";

        Pattern p = Pattern.compile("\\b[aeiouáéíóú][a-záéíóúñ]*", Pattern.CASE_INSENSITIVE);

        Matcher m = p.matcher(texto);

        while (m.find()) {

            System.out.println(m.group());

        }

    }

}
```

3. Escribe un programa que oculte los números de teléfono (9 dígitos) de un texto sustituyéndolos por "[OCULTO]", usando `replaceAll` con la regex adecuada. (2 ptos)

RESPUESTA / SOLUCIÓN

```
public class OcultarTelefonos {

    public static void main(String[] args) {

        String texto = "Llama al 612345678 o al 699000111 hoy";

        String resultado = texto.replaceAll("\\d{9}", "[OCULTO]");

        System.out.println(resultado);

        // Llama al [OCULTO] o al [OCULTO] hoy

    }

}
```

Parte 4: Pregunta teórica

2 puntos

1. Explica la diferencia entre `matches()` y `find()`, y para qué sirven los grupos de captura. Pon un ejemplo de uso de grupo.

RESPUESTA / SOLUCIÓN

`matches()` exige que toda la cadena encaje con la regex; `find()` localiza coincidencias parciales y puede llamarse repetidamente para recorrerlas.

Los grupos de captura (paréntesis) permiten extraer partes de la coincidencia: en la regex `(\d{2})/(\d{2})/(\d{4})` para una fecha, `group(1)` sería el día, `group(2)` el mes y `group(3)` el año.

Ejemplo: `Pattern.compile("Fecha: (\d{2})/(\d{2})/(\d{4})").matcher(texto).find()` permite recuperar día/mes/año sin re-parsing manual.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)